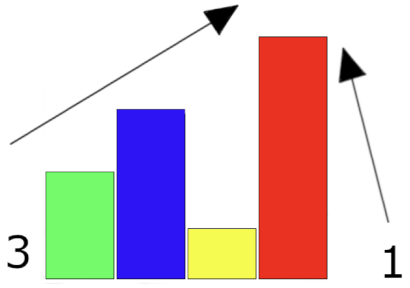
Chapter IV

Subject

	Exercise 00
Rush01	
Turn-in directory: *ex*00/	
Files to turn in: `All the necessary files (*.c)`	
Allowed functions: `write, malloc, free`	

- Your source code will be compiled as follows: `cc -Wall -Wextra -Werror -o rush01 *.c`.
- Your submission directory must contain all files required to compile your program.
- Create a program that solves the following problem:
  Given a 4x4 grid, place boxes of heights 1 to 4 on each available cell so that every row and column sees the correct number of boxes from each possible point of view (left/right for rows, top/bottom for columns).
- Example for one row or one column: The box of height 3 will hide the box of height 1 from the left, so there are 3 visible boxes. Only one box is visible from the right, as the box of height 4 hides everything.



- Each view (2 per row and 2 per column) will have a given value. Your program must place the boxes correctly, ensuring that each row and column contains only one box of each size.

- Your output must display the first solution you encounter.
- Here is how we will launch your program:

```
> ./rush01 "col1top col2top col3top col4top col1bottom col2bottom col3bottom col4bottom row1left
    row2left row3left row4left row1right row2right row3right row4right"
```

- "col1top" represents the value for the left column upper point of view, etc.
  Refer to appendix 1 to see what represents each element.
- Each element of the string is a number ranging between '1' and '4'.
- This is the only acceptable input for your program. Any other input must be considered an error.
- Here is an example of intended input/output for a valid set.

```
./rush01 "4 3 2 1 1 2 2 2 4 3 2 1 1 2 2 2" | cat -e
1 2 3 4$
2 3 4 1$
3 4 1 2$
4 1 2 3$
```

- Refer to appendix 2 and 3 for a flat vision, and appendix 4 for a 3D vision.
- In case of an error or if you cannot find any solutions, display "Error" followed by a newline.
- If you want bonus points, you may try to handle other map sizes (up to 9x9).
- As usual, if a bonus works but the mandatory one fails the tests, you will receive a score of 0.

Chapter V

Appendix

What follows is an artistic view of your program. You need to submit a program as described in the previous chapter.
The sole purpose of these representations is to help you understand the project.

V.1 Appendix 1

	col1top	col2top	col3top	col4top	
row1left					row1right
row2left					row2right
row3left					row3right
row4left					row4right
	col1bottom	col2bottom	col3bottom	col4bottom	

Representation of the different points of vision: col`X`top, col`X`bottom, row`X`left and row`X`right.

V.2 Appendix 2

	4	3	2	1	
4					1
3					2
2					2
1					2
	1	2	2	2	

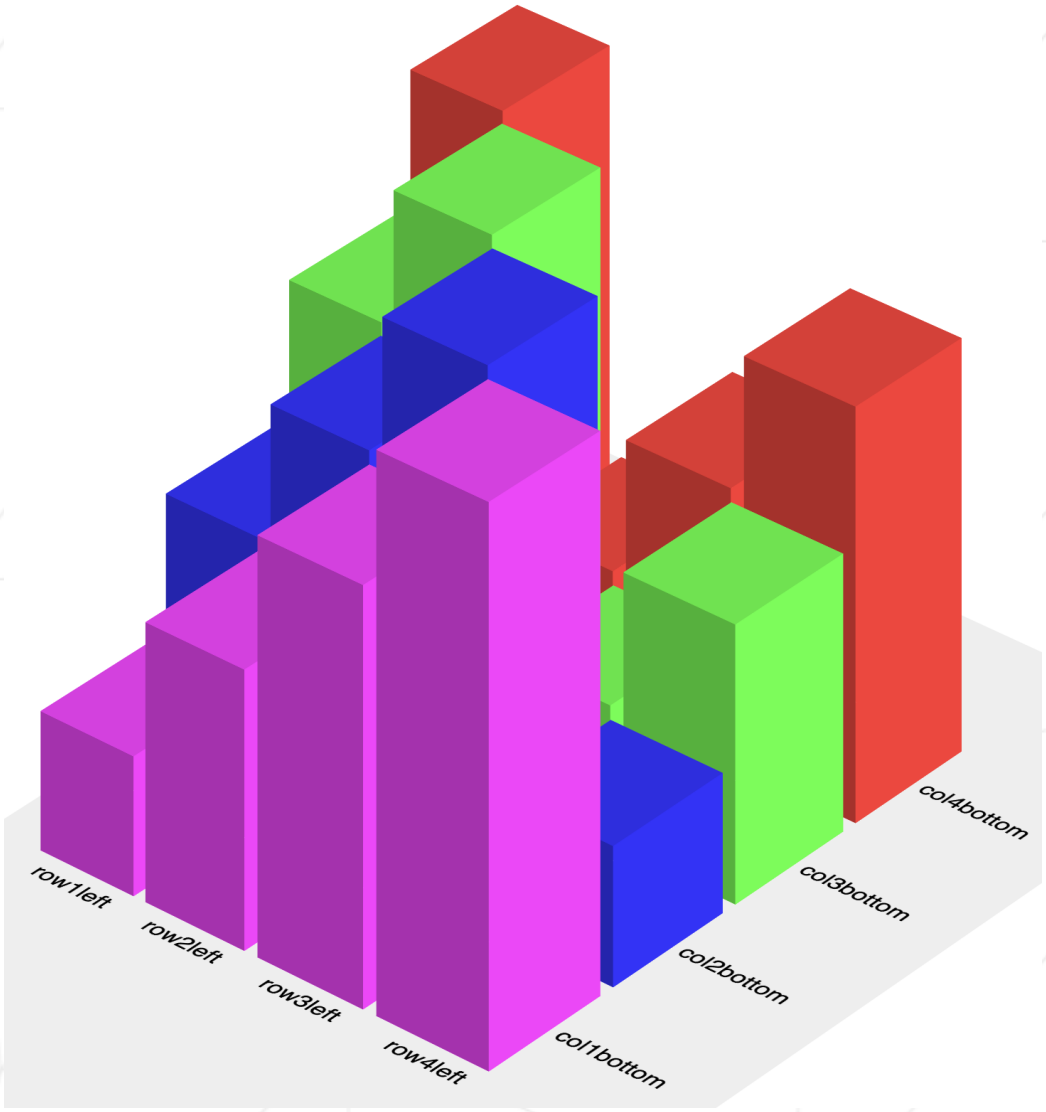
With assigned values for col* and row*, we get this.

V.3 Appendix 3

	4	3	2	1	
4	1	2	3	4	1
3	2	3	4	1	2
2	3	4	1	2	2
1	4	1	2	3	2
	1	2	2	2	

Your program must fill in the blanks of the 4x4 grid using the rules described above in this document.

V.4 Appendix 4



A 3D view of the 4x4 grid and the blocs.